

กลุ่ม [3] — [Variables, Mutability & Data Types]

1. [นายนาธาน ศรีนาการ](660710606) — Concept + Short Code Illustration
2. [นายภัทรพล ก่อมงคลกุล](660710613) — Detailed Code + Live Demo
3. [นายพีรตน์ นิกพงษ์] (660710624) — Rust vs Other Language + PPL Analysis
4. [นายรัชต กระเช้าเฟี้ยว] (660710626) — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1 (นายนาธาน ศรีนาการ)

**Concept + Short Code
Illustration**

แนวคิดหลัก (let, let mut, const, Shadowing) และ Learning Objectives

~5 นาที

2

Member 2 (นายภัทรพล ก่อมงคลกุล)

Detailed Code + Live Demo

อธิบายโค้ดเชิงลึก (Scalar/Compound Types, Integer Overflow) และสาธิต Live Demo สด

~5 นาที

3

Member 3 (นายพีรตน์ นิกพงศ์)

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา (Rust vs P)เปรียบเทียบภาษา (Rust vs Python) และวิเคราะห์เชิง Programming Language

~5 นาที

4

Member 4 (นายรัชต กระเช้าเพชร)

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย 3 กรณี, เฉลยแบบฝึกหัด และโจทย์ท้าทายผู้ฟัง

~5 นาที

[Variables, Mutability & Data Types] คืออะไร? (Concept)

- [ความหมาย/ที่มาของแนวคิด] ตัวแปรคือชื่อที่ผูก (bind) กับค่าในหน่วยความจำ โดย Rust กำหนดให้ตัวแปรเป็น immutable โดยค่าเริ่มต้นหลังกำหนดค่าแล้วเปลี่ยนไม่ได้
- [เหตุผลที่ Rust ออกแบบแนวคิดนี้] ช่วยให้โค้ดอ่านง่าย อ่านแล้วเดาพฤติกรรมได้ทันที ลด side effect จากการถูกแก้ไขค่าโดยไม่ตั้งใจ
- [ปัญหาที่แนวคิดนี้ช่วยแก้ไข] ช่วยป้องกัน Data Race และข้อผิดพลาดจากการเปลี่ยนค่าแบบไม่ตั้งใจตั้งแต่ขั้นตอน Compile Time
- [จุดสำคัญที่ผู้ฟังต้องจำ] ใช้ let ประกาศตัวแปร immutable, ใช้ let mut หากต้องการแก้ไขค่า, const ต้องระบุ Type เสมอ และ

```
const MAX_POINTS: u32 = 100_000;

fn main() {
    let x = 5;                // 1) Immutable variable
    let mut count = 0;        // 2) Mutable variable
    count += 1;
    let spaces = "   ";       // 3) Shadowing: เปลี่ยน Type จาก &str เป็น
                               usize
    let spaces = spaces.len();
    println!("x = {x}, count = {count}, spaces = {spaces}, max =
{MAX_POINTS}");
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
x = 5, count = 1, spaces = 3, max = 100000
```

Detailed Code + Live Demo

```
fn main() {
    // 1. Scalar Types (i32, u8, char 4 bytes Unicode)
    let a: i32 = -42;
    let b: u8 = 255;
    let ch: char = '🔥';
    // 2. Compound Types (Tuple & Array)
    let t: (i32, f64, char) = (500, 6.4, 'R');
    let (x, y, z) = t;           // Destructuring
    let arr: [i32; 5] = [1, 2, 3, 4, 5]; // Stack-allocated fixed length
    let zeros = [0; 3];
    println!("Scalar: i32={a}, char={ch} (size={} bytes)",
std::mem::size_of::<char>());
    println!("Tuple: {:?}, t.0={}, destructured: x={x}, y={y}, z={z}", t, t.0);
    println!("Array: {:?}, len={}, zeros={:?}", arr, arr.len(), zeros);
    // 3. Safe Integer Overflow Handling
    println!("255 wrapping_add 1 = {}", b.wrapping_add(1)); // วนกลับเป็น 0
    println!("255 checked_add 1 = {:?}", b.checked_add(1)); // คำนวณค่า None
}
```

Demo Checklist (ทำไว้เป็น note ของ slide แทน)

- ☐ [ขั้นตอน Demo ที่ 1](Variables & Shadowing): รัน cargo run แสดงความต่างของ let, let mut และ Shadowing ใน Block Scope
- ☐ [ขั้นตอน Demo ที่ 2] (Data Types): แสดงผล Scalar (i32, u8, char ขนาด 4 ไบต์) และ Compound Types (Tuple, Array)
- ☐ [ขั้นตอน Demo ที่ 3] (Compile Errors): สาธิตการแก้ค่าตัวแปร Immutable (x = 6) และการใช้ mut พยายามเปลี่ยน Type เพื่อชี้ให้เห็น Error [E0384] และ [E0308]
- ☐ [สิ่งที่ต้องระวังระหว่าง Demo](Integer Overflow) ใน Debug Build จะทำให้โปรแกรม Panic! ต้องอธิบายการใช้ wrapping_add และ checked_add เพื่อจัดการอย่างปลอดภัย

Rust vs Python

Aspect	Rust	Python
Syntax	let x: i32 = 5; (ต้องมี let และระบุ Type ชัดเจน)	x = 5 กำหนดค่าได้ทันที
Semantics / Behavior	ตัวแปร immutable เป็นค่าเริ่มต้น ต้องระบุ mut หากจะแก้ค่า	ตัวแปรเปลี่ยนค่าได้เสมอ (rebind)
Type System	Static, Strong, มี Type Inference ตระกูล Hindley–Milner	Dynamic, strong (type ผูกกับค่า ไม่ใช่ตัวแปร)
Memory Management	Stack / Scope-based (ปราศจาก Garbage Collector)	Garbage collection + reference counting
Safety	จับ Type Error และ Mutation Error ทั้งหมดตั้งแต่ Compile Time	Type error ส่วนใหญ่พบตอน runtime และมี GC ช่วยจัดการ memory

Analysis: Python เลือกเน้น Performance สูงสุดและควบคุม Hardware โดยตรงผ่าน Manual Memory Management (malloc/free) ซึ่งเสี่ยงต่อ Memory Leak และ Buffer Overflow ส่วน Rust เลือกใช้ระบบ Borrowing Checker เข้ามาควบคุมความปลอดภัยตั้งแต่ Compile Time ผลลัพธ์คือได้โปรแกรมที่เร็วและประหยัดทรัพยากรเทียบเท่า C แต่ปลอดภัยจากข้อผิดพลาดทางหน่วยความจำโดยสมบูรณ์

Rust vs C

Aspect	Rust	C
Syntax	<code>let x: i32 = 5;</code> (ต้องมี <code>let</code> และระบุ Type ชัดเจน)	<code>int x = 5;</code> ต้องระบุชนิดข้อมูล
Semantics / Behavior	ตัวแปร <code>immutable</code> เป็นค่าเริ่มต้น ต้องระบุ <code>mut</code> หากจะแก้ค่า	ตัวแปรเปลี่ยนค่าได้เป็นค่าเริ่มต้น
Type System	Static, Strong, มี Type Inference ตระกูล Hindley–Milner	Static, มี implicit conversions ได้หลายกรณี
Memory Management	Stack / Scope-based (ปราศจาก Garbage Collector)	Manual memory management (<code>malloc</code> / <code>free</code>)
Safety	จับ Type Error และ Mutation Error ทั้งหมดตั้งแต่ Compile Time	ผู้พัฒนาต้องระวัง <code>pointer</code> , <code>buffer overflow</code> และ <code>memory leak</code>

Analysis: C เลือกเน้น Performance สูงสุดและความคุม Hardware โดยตรงผ่าน Manual Memory Management (`malloc/free`) ซึ่งเสี่ยงต่อ Memory Leak และ Buffer Overflow ส่วน Rust เลือกใช้ระบบ Borrowing Checker เข้ามาควบคุมความปลอดภัยตั้งแต่ Compile Time ผลลัพธ์คือได้โปรแกรมที่เร็วและประหยัดทรัพยากรเทียบเท่า C แต่ปลอดภัยจากข้อผิดพลาดทางหน่วยความจำโดยสมบูรณ์

Rust vs Java

Aspect	Rust	Java
Syntax	<code>let x: i32 = 5;</code> (ต้องมี <code>let</code> และระบุ Type ชัดเจน)	<code>int x = 5;</code> ต้องระบุชนิดข้อมูล
Semantics / Behavior	ตัวแปร <code>immutable</code> เป็นค่าเริ่มต้น ต้องระบุ <code>mut</code> หากจะแก้ค่า	ตัวแปรเปลี่ยนค่าได้ เว้นแต่ใช้ <code>final</code>
Type System	Static, Strong, มี Type Inference ตระกูล Hindley–Milner	Static, strong, มี type checking ตอน compile
Memory Management	Stack / Scope-based (ปราศจาก Garbage Collector)	Garbage Collector (GC)
Safety	จับ Type Error และ Mutation Error ทั้งหมดตั้งแต่ Compile Time	Type checking ตอน compile และมี GC ช่วยลด memory errors Type checking ตอน compile และมี GC ช่วยลด memory errors

Analysis: Java เน้นความปลอดภัยผ่าน Type Checking และจัดการ Memory ด้วย Garbage Collector ซึ่งมี Performance Overhead และ GC Pause ส่วน Rust เลือกตรวจสอบ Memory Safety ผ่านระบบ Ownership และ Scope-based ตั้งแต่ Compile Time ผลลัพธ์คือได้ความปลอดภัยระดับสูง ประมวลผลได้เร็วกว่า และใช้ Memory น้อยกว่าโดยไม่ต้องพึ่งพา GC

PPL Perspective — มุมมองเชิง Programming Language

Syntax

Rust ใช้คีย์เวิร์ด `let` สำหรับ Variable Declaration โดยระบุ Type Annotation หลังชื่อตัวแปรด้วย : เช่น `let x: i32` ซึ่งตรงกับภาษาสมัยใหม่อย่าง Kotlin/Swift/TypeScript

Semantics

`let` คือ Name Binding ผูกชื่อเข้ากับค่า ความหมายของ Immutability คือห้าม Rebind ค่าเดิม ส่วน Shadowing คือการสร้าง Lexical Binding ใหม่ใน Scope

Type System

Statically Typed และ Strongly Typed ปราศจาก Implicit Coercion (ไม่แปลง Numeric Type อัตโนมัติ) และมีระบบ Type Inference

Memory / Resource

ค่า Scalar, Tuple และ Array ขนาดแน่นอนเก็บบน Stack ตัวแปร Scalar ทั้งหมดมี Copy trait คั่นทรัพยากรอัตโนมัติเมื่อออกจาก Scope (Scope-based Resource Management)

Abstraction

แนวคิด Immutability by Default ยืมมาจาก Functional Programming เพื่อลด Side Effects

Why Rust?

การตรวจสอบทุกอย่างตั้งแต่ Compile Time ช่วยจับข้อผิดพลาดล่วงหน้า ป้องกัน Data Race และเป็นรากฐานของระบบ Ownership / Borrowing

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
fn main() {
    // 1. Assign ค่าซ้ำให้ตัวแปร immutable
    let x = 5;
    x = 6; // ✗ error[E0384]: cannot assign twice to immutable variable `x`

    // 2. ใช้ mut พยายามเปลี่ยน type
    let mut spaces = " ";
    spaces = spaces.len(); // ✗ error[E0308]: mismatched types

    // 3. บวกลบเลขต่าง type โดยไม่ cast
    let a: i32 = 5;
    let b: i64 = 10;
    let total = a + b; // ✗ error[E0308]: mismatched types
}
```

✓ Correct

```
fn main() {
    // 1. ระบุ let mut เมื่อต้องการเปลี่ยนค่า
    let mut x = 5;
    x = 6;

    // 2. ใช้ shadowing เมื่อต้องการเปลี่ยน type
    let spaces = " ";
    let spaces = spaces.len();

    // 3. แปลง type อย่างชัดเจนด้วย as
    let a: i32 = 5;
    let b: i64 = 10;
    let total = a as i64 + b;
}
```

ทำไมถึงผิด? (Why?)

1. ตัวแปรใน Rust เป็น immutable โดยค่าเริ่มต้น ต้องระบุ mut หากต้องการเปลี่ยนค่า
2. คีย์เวิร์ด mut อนุญาตให้เปลี่ยนเฉพาะ "ค่า" แต่เปลี่ยน Data Type ไม่ได้ หากต้องการเปลี่ยน Type ต้องใช้ Shadowing
3. Rust ไม่มี Implicit Numeric Conversion เพื่อป้องกันข้อมูลสูญหาย โปรแกรมเมอร์ต้องทำ Explicit Conversion ด้วย as เอง

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Challenge Problem 1)

กำหนดราคาสินค้าเป็นข้อความ "1200" ให้แปลงเป็น f64 แล้วลดราคา 10% จากนั้นบวกภาษีมูลค่าเพิ่ม (VAT) 7% และแสดงราคาสุทธิทศนิยม 2 ตำแหน่ง โดยใช้ชื่อตัวแปร price เพียงชื่อเดียว (ใช้ Shadowing) และห้ามใช้ mut



Hint

ใช้ `.parse::<f64>().unwrap()` แปลงข้อความ, ลด 10% (คูณ 0.9), บวก VAT 7% (คูณ 1.07), จัดรูปแบบทศนิยมด้วย `{price:.2}`

```
fn main() {  
    let price = "1200";  
    let price: f64 = price.parse().unwrap();  
    let price = price * 0.9;  
    let price = price * 1.07;  
    println!("ราคาสุทธิ = {price:.2} บาท"); // Output: 1155.60  
    บาท
```

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Challenge Problem 2)

หาอุณหภูมิต่ำสุดและสูงสุดจาก Array [31, 28, 35, 30, 33, 27] เก็บผลลัพธ์ไว้ใน Tuple (i32, i32) พร้อมคำนวณช่วงอุณหภูมิ (Max - Min)



Hint

สร้างตัวแปร `mut min, max = temps[0]`, วนลูป `for t in temps` อัปเดตค่าด้วย `if`, เก็บใส่ Tuple (min, max), และเข้าถึงสมาชิกนำมาลบกันด้วย `range.1 - range.0`

```
let temps: [i32; 6] = [31, 28, 35, 30, 33, 27];
let (mut min, mut max) = (temps[0], temps[0]);
for t in temps {
    if t < min { min = t; }
    if t > max { max = t; }
}
let range: (i32, i32) = (min, max);
println!("Min = {}, Max = {}, ช่วงอุณหภูมิ = {}", range.0,
range.1, range.1 - range.0);
}
```

Key Takeaways — สรุปสิ่งที่ควรจำ

- 1 **[Immutability by Default]** ตัวแปรปรับเปลี่ยนค่าไม่ได้โดยค่าเริ่มต้นเพื่อความปลอดภัย หากต้องการแก้ไขต้องระบุ `mut` ชัดเจน
- 2 **[Shadowing vs Mutability]** `mut` เปลี่ยนได้เฉพาะค่า แต่เปลี่ยน Data Type ไม่ได้ ส่วน Shadowing (`let` ชื่อเดิม) คือการสร้างตัวแปรใหม่จึงเปลี่ยน Type และ Scope ได้
- 3 **[Strong & Static Typing]** ไม่มี Implicit Numeric Conversion การแปลง Data Type ข้ามชนิดต้องทำอย่างชัดเจน (Explicit Conversion) ด้วย `as` เสมอ
- 4 **[PPL Foundation]** การผูกชื่อ (Binding), Lexical Scope และการจัดการหน่วยความจำแบบ Scope-based บน Stack (ไม่มี GC) เป็นรากฐานสำคัญของระบบ Ownership ใน Rust

References — เอกสารอ้างอิง

- The Rust Programming Language (The Rust Book) — Variables and Mutability & Data Types: <https://doc.rust-lang.org/book/>
- Rust by Example — Variable Bindings / Primitives: <https://doc.rust-lang.org/rust-by-example/>
- The Rust Standard Library — (Primitive Types) : <https://doc.rust-lang.org/std/#primitives>
- The Rust Reference — <https://doc.rust-lang.org/reference/>

GitHub Repository: <https://github.com/soonklang/rust-tutorial-2569>

Questions & Answers

กลุ่ม [03] — [Variables, Mutability & Data Types]

ขอขอบคุณอาจารย์และเพื่อนๆ ทุกคนที่รับฟัง